Scientific Forgery Image Detection

By: Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

May 8, 2026

Software Engineering Tools, CS 3338

Table of Contents

1. Introduction

1.1 Purpose

This document defines the requirements for the Scientific Forgery Image Detection System. The system is designed to detect copy-move forgery in biomedical and scientific images, where regions of an image are duplicated to manipulate research results.

The system allows users to upload scientific images through a web interface. The images are processed by a backend detection system that identifies and highlights potentially duplicated regions, supporting researchers and reviewers in identifying manipulated figures.

1.2 Intended Audience

This document is intended for:

- Software developers working on the system
- Course instructors evaluating the project
- Students involved in system design
- Future maintainers of the software

1.3 Overall Description

The system is a complete web-based application consisting of:

- A frontend interface for image uploads and displaying results
- A backend API for handling image processing requests
- A detection system capable of identifying potentially duplicated regions in biomedical images

At this stage (Snapshot 4), the system includes:

- Refined frontend image upload functionality
- Improved backend API integration
- Final fraudulent region detection workflow
- Enhanced suspicious region result display
- Improved error handling for invalid uploads
- Final interface and workflow improvements

2. External Interface Requirements

2.1 User Interface Requirements

The system provides a refined web interface with the following components:

- Homepage with project description
- Image upload page with improved layout
- File selection input for biomedical images
- Submit button for analysis requests
- Image preview functionality
- Detection results display section with highlighted regions
- Clear status messages (forgery detected / no forgery / error)
- Loading indicator during analysis

2.2 Software Interface Requirements

Frontend:

- React.js / HTML
- HTML/CSS
- JavaScript

Backend:

- Python
- FastAPI
- Uvicorn
- OpenCV
- NumPy
- Pillow

Deployment:

- Docker
- Docker Compose

Communication:

- HTTP requests between frontend and backend
- JSON-based API responses
- Base64 encoded result images

2.3 Hardware Interface Requirements

No specialized hardware is required. The system runs on standard computing devices capable of running a web browser and Docker containers.

2.4 Communication Requirements

- REST API communication between frontend and backend

- Biomedical image upload through HTTP requests
- Detection analysis responses returned in JSON format
- Frontend display of suspicious region analysis results
- Improved error response formatting for invalid uploads

3. Legal and Ethical Considerations

3.1 Data Integrity

The system processes scientific images, which may contain sensitive or research-related content. All uploaded images are handled securely and only used for analysis purposes. Uploaded images are removed from the server immediately after analysis.

3.2 Privacy Considerations

- Uploaded images are used only for detection purposes
- No user personal data is required
- Uploaded images are deleted from the server after analysis
- Data should not be shared externally without permission

3.3 Ethical Considerations

- The system is intended to support scientific integrity and image analysis research
- Detection results should not be considered definitive proof of misconduct
- False positives or false negatives may occur due to limitations in early-stage detection methods
- Human review is still necessary when evaluating scientific images

4. Glossary

Term	Definition
API	Interface used for communication between software components
Backend	Server-side system that processes requests
Frontend	User interface of the application
Copy-Move Forgery	Image manipulation technique where parts of an image are duplicated
FastAPI	Python framework for building APIs
React.js	JavaScript library for building user interfaces
HTTP	Protocol used for web communication
JSON	Lightweight data format used for API responses

OpenCV	Open-source computer vision library used for image analysis
ORB	Oriented FAST and Rotated BRIEF — keypoint detection algorithm
Keypoint	A distinctive location in an image used for matching
Docker	Platform for containerizing applications
Docker Compose	Tool for defining and running multi-container Docker applications

5. Versions Table

Version	Date	Description	Authors
1.0	May 8, 2026	Initial SRS created for Snapshot 1	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.1	May 10, 2026	Updated frontend implementation progress and revised interface requirements	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.2	May 12, 2026	Added fraudulent region detection requirements and backend/frontend integration updates	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.3	May 14, 2026	Added final workflow improvements, detection result refinements, and finalized system requirements	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

6. References

- FastAPI Documentation: https://fastapi.tiangolo.com/
- React Documentation: https://react.dev/
- OpenCV Documentation: https://opencv.org/
- Docker Documentation: https://docs.docker.com/
- OWASP API Security Guidelines: https://owasp.org/www-project-api-security/